

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Case Study

COURSE NAME: Cloud Computing and Big Data Analytics

COURSE CODE: CSA15

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)*

Dave's Taxonomy: Precision (Level 3) **Total Marks: 30**
Question Count: 3

Code of Conduct

I M.Devi Srilatha (192525236) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Tasks

Case Study

1. Case Study 1: Smart Retail Inventory Management System

A retail chain implements a cloud-based inventory system connected to store devices. Clients (POS systems and dashboards) communicate via APIs to centralized cloud services. Cloud storage stores product data, transaction logs, and analytics datasets. The system uses platform services for analytics and demand forecasting. Secure access is maintained through VPNs, firewalls, and API gateways. This solution integrates infrastructure, services, and web applications for efficient operations.

2. Case Study 2: Cloud-Based Document Collaboration System

An enterprise deploys a document collaboration tool similar to Google Docs. Users access the platform via browsers, enabling real-time editing and sharing. Documents are stored in distributed cloud storage with version control mechanisms. Web APIs enable synchronization across multiple clients and devices. Network infrastructure ensures low latency and high availability globally. Security includes encryption, identity management, and access permissions for collaboration.

3. Case Study 3: Online Food Ordering Platform

A startup builds a cloud-based food ordering system accessible via web browsers and mobile clients.

Frontend clients interact with backend services through RESTful Web APIs hosted on a cloud platform.

Cloud storage is used to store menu images, invoices, and customer data securely. The infrastructure uses virtual machines and managed Kubernetes for scalability and reliability. Security is enforced through authentication tokens, HTTPS, and role-based access control. The system demonstrates integration of clients, APIs, storage, and platform services in real time.

Case Study 1

Smart Retail Inventory Management System

1. Introduction

A **Smart Retail Inventory Management System** is a database-based system used to manage products, suppliers, customers, purchases, sales, and stock levels in a retail store. It replaces manual inventory management with an automated system.

The system keeps track of available stock and updates inventory whenever products are purchased or sold. It also helps identify products that are running low and need to be reordered.

2. Problem Statement

Managing inventory manually can result in:

- Incorrect stock information
- Duplicate records
- Difficulty tracking sales and purchases
- Stock shortages or excess stock
- Manual calculation errors
- Difficulty generating reports

Therefore, a computerized inventory management system is required to maintain accurate and updated stock information.

3. Objectives

The main objectives are:

1. To maintain product and supplier information.
2. To record purchases and sales.
3. To automatically update stock levels.
4. To identify low-stock products.
5. To reduce manual errors and data duplication.
6. To generate inventory and sales reports.
7. To help management make better purchasing decisions.

4. Main Entities

The important entities are:

- **PRODUCT** – stores product details, price, and stock quantity.
- **CATEGORY** – stores product categories.

- **SUPPLIER** – stores supplier information.
- **CUSTOMER** – stores customer details.
- **EMPLOYEE** – stores employee information.
- **PURCHASE** – records products purchased from suppliers.
- **PURCHASE_DETAIL** – stores individual products in a purchase.
- **SALE** – records customer sales.
- **SALE_DETAIL** – stores individual products in a sale.

5. Relationships

The major relationships are:

- One **Category** can have many Products.
- One **Supplier** can have many Purchases.
- One **Purchase** can contain many Products.
- One **Customer** can have many Sales.
- One **Sale** can contain many Products.
- One **Employee** can process many Sales and Purchases.

Thus, the system mainly contains **one-to-many (1:N)** relationships.

6. Relational Schema

The database can be represented using the following relations:

CATEGORY

(Category_ID, Category_Name, Description)

SUPPLIER

(Supplier_ID, Supplier_Name, Contact_Number, Email, Address)

PRODUCT

(Product_ID, Product_Name, Category_ID, Supplier_ID, Unit_Price, Stock_Quantity, Reorder_Level)

CUSTOMER

(Customer_ID, Customer_Name, Phone, Email, Address)

EMPLOYEE

(Employee_ID, Employee_Name, Role, Phone, Email)

PURCHASE

(Purchase_ID, Supplier_ID, Employee_ID, Purchase_Date, Total_Amount)

PURCHASE_DETAIL

(Purchase_ID, Product_ID, Quantity, Purchase_Price)

SALE

(Sale_ID, Customer_ID, Employee_ID, Sale_Date, Total_Amount)

SALE_DETAIL

(Sale_ID, Product_ID, Quantity, Selling_Price)

7. Functional Dependencies

Some important functional dependencies are:

Product_ID → Product_Name, Category_ID, Supplier_ID, Unit_Price, Stock_Quantity, Reorder_Level

Category_ID → Category_Name, Description

Supplier_ID → Supplier_Name, Contact_Number, Email, Address

Customer_ID → Customer_Name, Phone, Email, Address

Employee_ID → Employee_Name, Role, Phone, Email

Sale_ID → Customer_ID, Employee_ID, Sale_Date, Total_Amount

Purchase_ID → Supplier_ID, Employee_ID, Purchase_Date, Total_Amount

For SALE_DETAIL:

(Sale_ID, Product_ID) → Quantity, Selling_Price

8. Normalization

The database can be normalized to reduce redundancy.

1NF

All attributes contain atomic values and there are no repeating groups.

2NF

The database is in 1NF and non-key attributes depend on the complete primary key.

3NF

The database is in 2NF and transitive dependencies are removed. For example, category information is stored separately in the CATEGORY table rather than repeatedly in PRODUCT.

Thus, the database design can satisfy **3NF** and provide efficient data management.

9. Inventory Process

The basic process is:

Purchase → Stock Increase → Product Sale → Stock Decrease → Low Stock Check → Reorder

For example, if the store has 50 units and purchases 20 more:

New Stock = 50 + 20 = 70 units

If 5 units are sold:

Remaining Stock = 70 – 5 = 65 units

When:

Stock_Quantity ≤ Reorder_Level

the system generates a low-stock alert.

10. Smart Features

The system can provide:

- Automatic stock updates
- Low-stock alerts
- Sales reports
- Inventory reports
- Barcode scanning
- Most-selling product analysis
- Demand prediction
- Automatic reorder recommendations

11. Advantages

The system:

1. Reduces manual work.
2. Improves stock accuracy.
3. Reduces data-entry errors.
4. Prevents stock shortages.
5. Helps avoid overstocking.
6. Provides faster reports.
7. Improves purchasing decisions.
8. Saves time and operational effort.

12. Future Enhancements

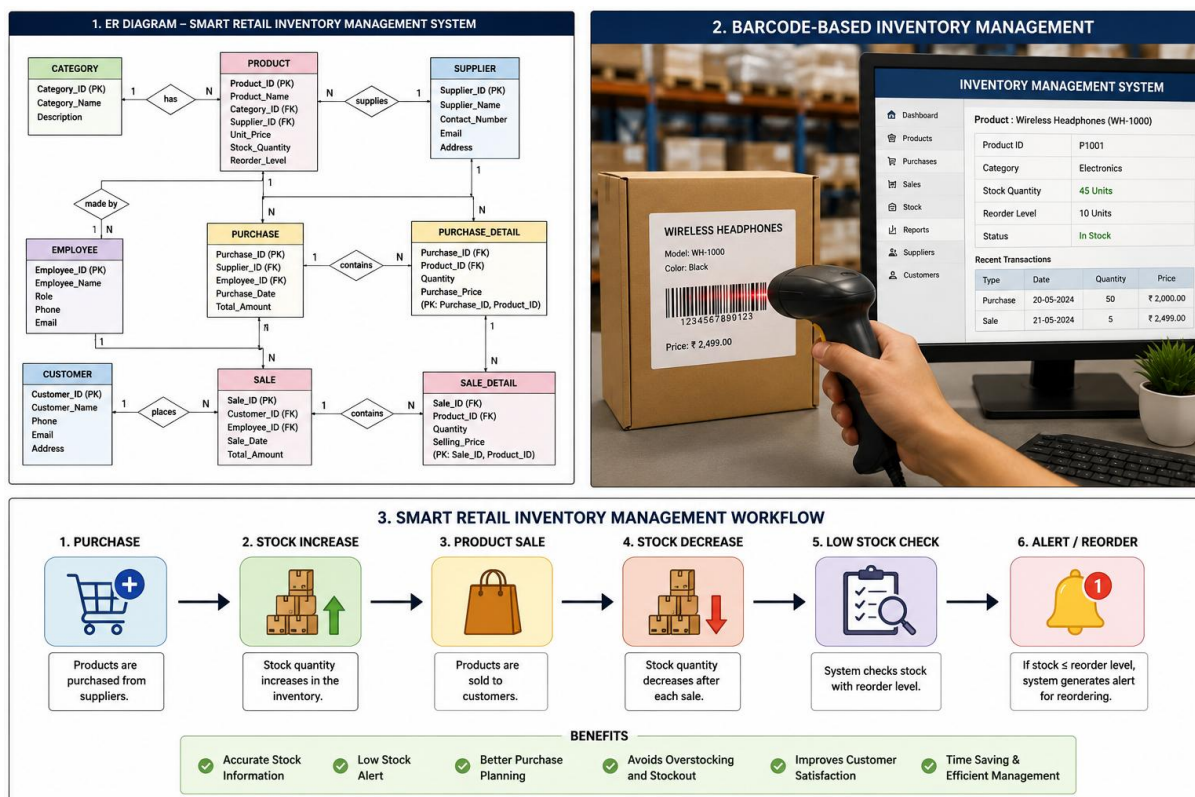
The system can be enhanced using:

- **Artificial Intelligence** for demand prediction.
- **Machine Learning** for sales forecasting.
- **Cloud Computing** for remote access.
- **Mobile Applications** for inventory monitoring.
- **Barcode/QR Code scanning** for faster transactions.
- **Automatic Reordering** when stock reaches the reorder level.

13. Conclusion

The **Smart Retail Inventory Management System** provides an efficient way to manage products, suppliers, customers, purchases, sales, and inventory. It reduces manual errors and keeps stock information updated.

By using database normalization, primary keys, foreign keys, and automated stock updates, the system provides reliable inventory management. Future integration with AI, machine learning, barcode scanning, and cloud technology can make the system more intelligent and suitable for modern retail businesses.



Case Study 2

Cloud-Based Document Collaboration System

1. Introduction

A **Cloud-Based Document Collaboration System** is a cloud computing platform that allows multiple users to create, edit, store, and share documents through the Internet. Examples of similar systems include Google Docs and Microsoft 365.

Users can access documents through web browsers from different computers, tablets, or mobile devices. Since the documents are stored in cloud storage, users do not need to keep separate copies on their local devices.

The system supports **real-time collaboration**, meaning multiple authorized users can work on the same document at the same time. Changes are synchronized across connected devices using web APIs and cloud services.

2. Problem Statement

Traditional document management can create several problems. Users may have multiple copies of the same document, making it difficult to identify the latest version. Sharing files through email or storage devices can also result in duplicate files, data loss, and synchronization problems.

Therefore, a cloud-based document collaboration system is required to:

- Store documents centrally in the cloud.
- Allow multiple users to edit documents.
- Synchronize changes in real time.
- Maintain different versions of documents.
- Provide secure sharing and access control.
- Allow users to access documents from different devices.
- Provide high availability and reliable storage.

3. Objectives

The main objectives of the system are:

1. To provide cloud-based document storage.
2. To support real-time document editing.
3. To allow users to share documents securely.
4. To synchronize changes across multiple devices.
5. To maintain document version history.

6. To provide authentication and identity management.
7. To implement access permissions.
8. To protect documents using encryption.
9. To provide high availability.
10. To provide fast access to users from different geographical locations.

4. Main Components

The system consists of several important components.

4.1 Users

Users access the platform through web browsers or applications.

Users may have different roles such as:

- Administrator
- Document Owner
- Editor
- Viewer

4.2 Web Application

The web application provides the interface through which users can:

- Create documents
- Open documents
- Edit documents
- Share documents
- Download documents
- View previous versions

4.3 Cloud Storage

Documents are stored in distributed cloud storage rather than on a single local computer.

Distributed storage provides:

- Scalability
- Reliability
- Data availability
- Backup
- Fault tolerance

4.4 Version Control

Every major change to a document can be recorded as a new version.

For example:

Version 1 → Version 2 → Version 3 → Version 4

If a mistake occurs, an earlier version can be restored.

4.5 Web APIs

Web APIs allow different clients and cloud services to communicate.

For example:

Browser → API → Cloud Server → Storage

APIs can be used for document updates, authentication, sharing, and synchronization.

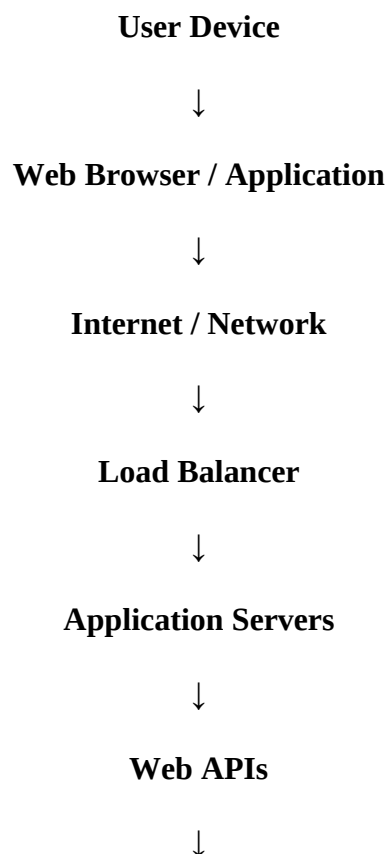
4.6 Network Infrastructure

The system requires a reliable network infrastructure to provide fast communication between users and cloud servers.

Content delivery networks, geographically distributed servers, and optimized network routing can help reduce latency for users in different locations.

5. System Architecture

The basic architecture can be represented as:



Document Collaboration Service



Distributed Cloud Storage + Database

The architecture allows multiple users to access the same document while the cloud services coordinate updates and storage.

6. Real-Time Collaboration

One of the most important features is real-time collaboration.

Suppose three users are editing the same document:

- User A edits the introduction.
- User B edits the conclusion.
- User C adds a table.

The system receives these changes and synchronizes them with the other connected users.

The general process is:

User Edit → API Request → Collaboration Server → Change Processing → Synchronization → Other Users

This allows users to see changes without repeatedly downloading and uploading the entire document.

Technologies such as **WebSockets** or similar real-time communication mechanisms can be used to maintain continuous communication between clients and servers.

7. Version Control

Version control maintains the history of changes made to documents.

For example:

Version	User	Action
V1	User A	Created document
V2	User B	Added content
V3	User A	Modified content
V4	User C	Added table

If the latest version contains an unwanted change, an earlier version can be restored.

This provides:

- Recovery from accidental changes
- Change tracking
- Document history
- Better collaboration

8. Security

Security is an important requirement because documents may contain confidential organizational information.

8.1 Authentication

Users must provide valid credentials before accessing the platform.

Examples include:

- Username and password
- Multi-factor authentication
- Single sign-on

8.2 Authorization

After authentication, the system determines what each user is allowed to do.

For example:

- **Owner:** Full control
- **Editor:** Can modify the document
- **Viewer:** Can only read the document

8.3 Encryption

Data should be protected using encryption.

Two important areas are:

Data in transit: Protected while moving between the user's device and cloud servers.

Data at rest: Protected while stored in cloud storage.

8.4 Access Permissions

Document owners can control who can access a document.

Permissions may include:

- View
- Comment
- Edit
- Share
- Download

This prevents unauthorized users from modifying sensitive information.

9. Cloud Computing Characteristics

The system demonstrates several important cloud computing characteristics.

On-Demand Access

Users can access documents whenever required through an Internet connection.

Resource Pooling

Cloud infrastructure shares computing and storage resources among many users.

Scalability

Storage and computing resources can be increased when the number of users grows.

High Availability

Multiple servers and storage systems can help keep the service available even if one component fails.

Broad Network Access

Users can access the system from different devices and locations.

10. High Availability and Fault Tolerance

A global document collaboration platform must continue operating even when hardware or software failures occur.

High availability can be achieved using:

- Multiple application servers
- Load balancing
- Distributed storage
- Database replication
- Backup systems
- Failover mechanisms

- Multiple geographical regions

If one server becomes unavailable, requests can be redirected to another available server.

This reduces downtime and improves reliability.

11. Database Design

The system can maintain information using several entities.

USER

USER(User_ID, Name, Email, Password_Hash, Role)

DOCUMENT

DOCUMENT(Document_ID, Title, Owner_ID, Created_Date, Modified_Date)

DOCUMENT_VERSION

DOCUMENT_VERSION(Version_ID, Document_ID, User_ID, Version_Number, Created_Date, Storage_Path)

PERMISSION

PERMISSION(Permission_ID, Document_ID, User_ID, Access_Level)

COMMENT

COMMENT(Comment_ID, Document_ID, User_ID, Comment_Text, Created_Date)

SESSION

SESSION(Session_ID, User_ID, Device_ID, Login_Time, Logout_Time)

12. Functional Dependencies

Some important functional dependencies are:

User_ID → Name, Email, Password_Hash, Role

Document_ID → Title, Owner_ID, Created_Date, Modified_Date

Version_ID → Document_ID, User_ID, Version_Number, Created_Date, Storage_Path

Permission_ID → Document_ID, User_ID, Access_Level

Comment_ID → Document_ID, User_ID, Comment_Text, Created_Date

These dependencies help organize the database efficiently.

13. Normalization

Normalization can be applied to reduce data redundancy.

1NF

All attributes contain atomic values and repeating groups are removed.

2NF

The database is in 1NF and partial dependencies are eliminated.

3NF

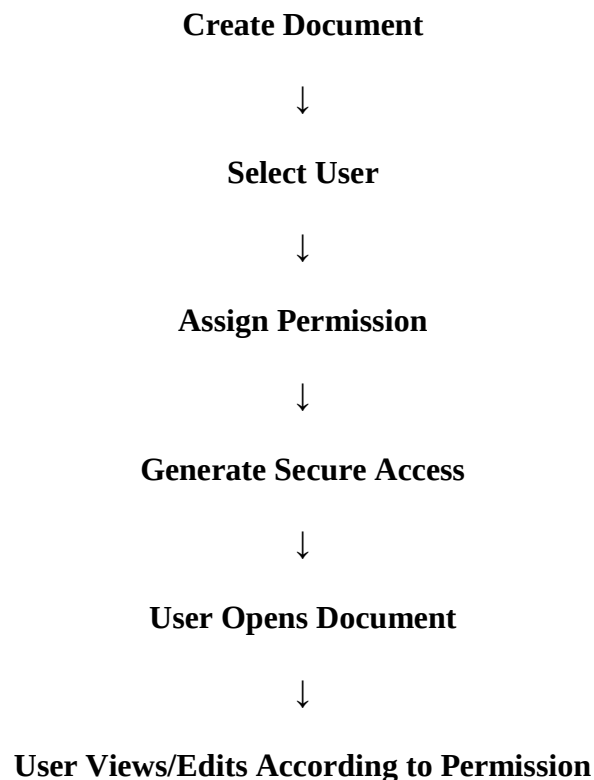
The database is in 2NF and transitive dependencies are removed.

For example, user information is stored in the USER table instead of repeatedly storing the user's name and email in every document record.

This improves consistency and reduces duplication.

14. Document Sharing Process

The document-sharing process can be explained as:



For example, a document owner can give:

- User A → Editor
- User B → Viewer
- User C → Commenter

This provides controlled collaboration.

15. Advantages

The system provides several benefits:

1. Access documents from anywhere.
2. Supports real-time collaboration.
3. Eliminates multiple uncontrolled copies.
4. Provides automatic synchronization.
5. Maintains version history.
6. Provides centralized cloud storage.
7. Supports secure document sharing.

16. Challenges

Some challenges associated with the system include:

- Dependence on Internet connectivity.
- Security and privacy risks.
- Managing simultaneous edits.
- Network latency.
- Large-scale storage requirements.
- Authentication and authorization management.
- Maintaining service availability.
- Handling failures across distributed systems.

17. Future Enhancements

The system can be enhanced by implementing:

Artificial Intelligence

AI can provide features such as document summarization, grammar assistance, and intelligent search.

Advanced Version Management

The system can provide detailed comparison between document versions.

Offline Editing

Users can edit documents without an active Internet connection and synchronize changes when connectivity is restored.

Automated Backup

Documents can be automatically backed up to multiple storage locations.

Advanced Security

Zero-trust security, stronger authentication, and advanced monitoring can improve protection.

Global Cloud Deployment

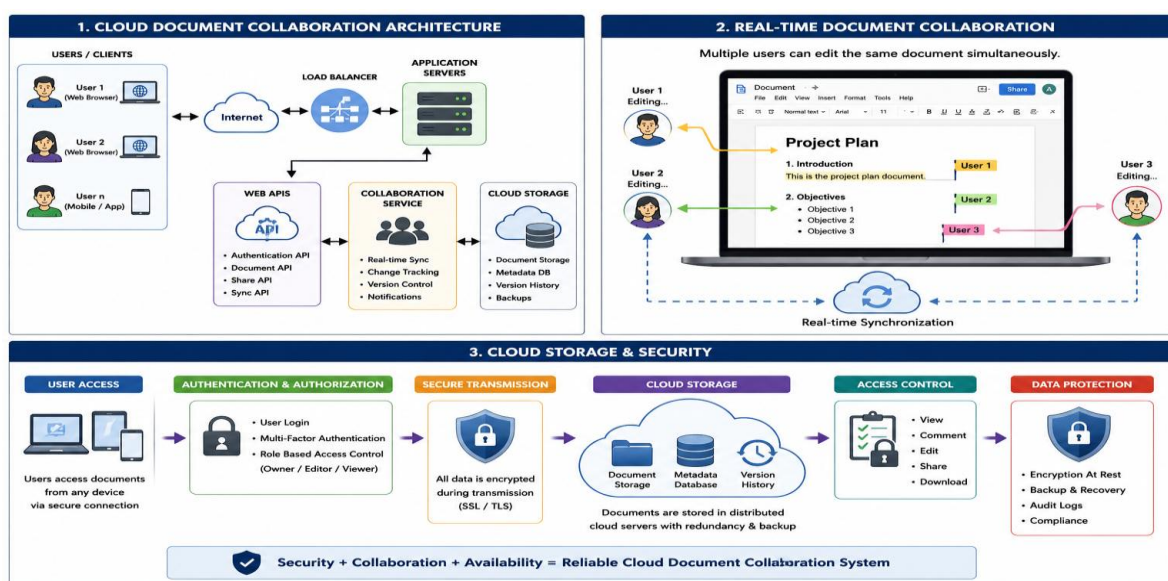
Deploying services in multiple geographical regions can reduce latency and improve availability.

18. Conclusion

The **Cloud-Based Document Collaboration System** provides a centralized and scalable platform for creating, storing, editing, and sharing documents. By using cloud storage, web APIs, distributed infrastructure, version control, and real-time synchronization, multiple users can collaborate efficiently from different locations and devices.

Security mechanisms such as authentication, authorization, encryption, and access permissions protect documents from unauthorized access. Distributed storage, load balancing, replication, and backup mechanisms improve availability and reliability.

Therefore, the system is an effective example of **cloud computing, distributed storage, real-time collaboration, network communication, and secure resource sharing** in a modern enterprise environment.



Case Study 3

Online Food Ordering Platform

1. Introduction

An **Online Food Ordering Platform** is a cloud-based application that allows customers to browse restaurants, view menus, place food orders, make payments, and track their orders using web browsers or mobile applications.

The frontend communicates with backend services through **RESTful Web APIs**. Cloud storage is used to securely store menu images, invoices, and customer-related information. Virtual machines and managed Kubernetes provide scalability and reliability when the number of users increases.

2. Problem Statement

Traditional food ordering methods may involve phone calls, manual order entry, and limited order tracking. These methods can cause incorrect orders, delays, and difficulty managing large numbers of customers.

The cloud-based platform provides a centralized system for:

- Browsing food menus.
- Placing orders online.
- Managing customer information.
- Processing orders efficiently.
- Storing invoices and menu images.
- Tracking orders.
- Supporting many users simultaneously.
- Providing secure access.

3. Objectives

The main objectives are:

1. To provide online food ordering through web and mobile clients.
2. To display restaurant menus and food items.
3. To allow customers to place and track orders.
4. To connect frontend clients with backend services using REST APIs.
5. To securely store customer and order information.
6. To provide scalability using virtual machines and Kubernetes.
7. To protect the system using authentication and HTTPS.
8. To control access using role-based access control.

4. Main Components

4.1 Frontend Clients

Customers access the platform through:

- Web browsers
- Android applications
- iOS applications

The frontend allows users to browse menus, add food to a cart, place orders, and view order status.

4.2 RESTful Web APIs

REST APIs act as the communication layer between frontend applications and backend services.

For example:

Mobile App → REST API → Backend Service → Database

APIs can handle:

- User login
- Menu retrieval
- Order creation
- Payment information
- Order tracking
- Customer details

4.3 Cloud Storage

Cloud storage is used to store:

- Menu images
- Customer documents
- Invoices
- Order-related files

Cloud storage provides scalability, availability, and backup capabilities.

4.4 Virtual Machines

Virtual machines provide computing resources for running application services. Additional virtual machines can be used when demand increases.

4.5 Managed Kubernetes

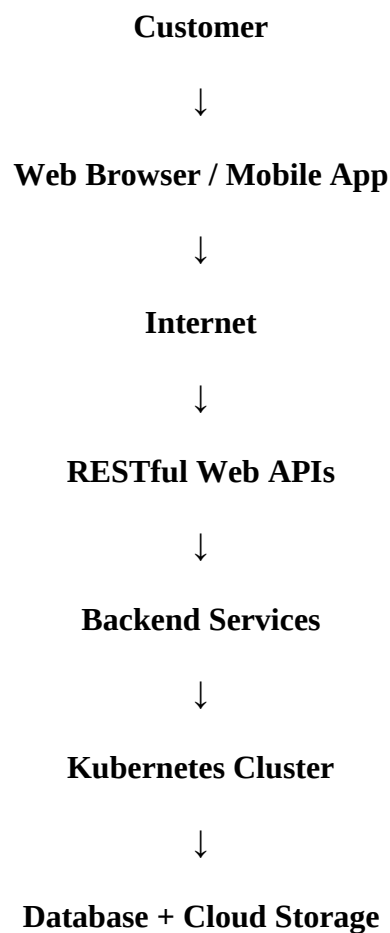
Kubernetes manages containers running different application services.

It provides:

- Automatic scaling
- Load balancing
- Service management
- Fault recovery
- High availability

5. System Architecture

The basic architecture is:



The frontend sends requests to APIs, and the backend processes the requests and communicates with databases and cloud storage.

6. Food Ordering Process

The basic ordering process is:

Login → Browse Restaurant → Select Food → Add to Cart → Place Order → Order Processing → Delivery → Order Completion

For example, when a customer selects a pizza, the application sends the order information to the backend through a REST API. The backend validates the request, stores the order, and sends the order status back to the customer.

7. Security

Security is important because the platform handles customer and transaction information.

Authentication Tokens

After successful login, the system provides an authentication token. The token is used to verify the user's identity when making API requests.

HTTPS

HTTPS protects communication between the user's device and cloud services by encrypting data during transmission.

Role-Based Access Control

Different users receive different permissions.

Role	Access
Customer	Browse menu and place orders
Restaurant Staff	Manage menu and orders
Delivery Staff	View assigned deliveries
Administrator	Manage the entire system

This prevents unauthorized users from accessing restricted functions.

8. Database Design

Important tables can include:

CUSTOMER

(Customer_ID, Name, Email, Phone, Address)

RESTAURANT

(Restaurant_ID, Restaurant_Name, Location, Contact)

MENU_ITEM

(Item_ID, Restaurant_ID, Item_Name, Price, Category, Image_URL)

ORDER

(Order_ID, Customer_ID, Restaurant_ID, Order_Date, Total_Amount, Status)

ORDER_DETAIL

(Order_ID, Item_ID, Quantity, Price)

PAYMENT

(Payment_ID, Order_ID, Payment_Date, Amount, Payment_Status)

DELIVERY

(Delivery_ID, Order_ID, Delivery_Person_ID, Delivery_Status)

9. Functional Dependencies

Some important functional dependencies are:

Customer_ID → Name, Email, Phone, Address

Restaurant_ID → Restaurant_Name, Location, Contact

Item_ID → Restaurant_ID, Item_Name, Price, Category, Image_URL

Order_ID → Customer_ID, Restaurant_ID, Order_Date, Total_Amount, Status

Payment_ID → Order_ID, Payment_Date, Amount, Payment_Status

These dependencies help organize the database and reduce data duplication.

10. Normalization

The database can be normalized up to 3NF.

- **1NF:** All attributes contain atomic values.
- **2NF:** Partial dependencies are removed.
- **3NF:** Transitive dependencies are removed.

For example, customer information is stored separately in the CUSTOMER table rather than being repeated for every order.

11. Scalability and Reliability

The platform may receive thousands of orders during peak periods. Cloud infrastructure helps handle this workload.

Virtual Machines provide computing resources.

Kubernetes manages containers and can automatically increase or decrease the number of running application instances based on demand.

For example:

Normal Traffic → Few Instances

High Traffic → More Instances

Traffic Decreases → Instances Reduced

This helps maintain performance while using resources efficiently.

12. Advantages

The system provides several benefits:

1. Online ordering from anywhere.
2. Faster order processing.
3. Real-time order tracking.
4. Secure customer information.
5. Easy menu management.
6. Cloud-based scalability.
7. High availability.
8. Reduced manual work.
9. Easy storage of invoices and images.
10. Better customer experience.

13. Future Enhancements

The platform can be improved using:

- **AI-based food recommendations**
- **Demand prediction**
- **Chatbots for customer support**
- **Real-time delivery tracking**
- **Personalized offers**
- **Automated restaurant recommendations**
- **Advanced analytics**
- **Serverless cloud services**

14. Conclusion

The **Online Food Ordering Platform** is a practical example of cloud computing that integrates frontend clients, RESTful Web APIs, backend services, cloud storage, virtual machines, and Kubernetes.

Customers can conveniently browse menus, place orders, and track deliveries through web or mobile applications. Security mechanisms such as authentication tokens, HTTPS, and role-based access control protect user and application data.

The use of virtual machines and managed Kubernetes provides **scalability, reliability, and high availability**, making the platform suitable for handling large numbers of users and real-time food orders.

